Proyecto de Análisis y Diseño de Software

Entrega 3.1.: Codificación y Pruebas de Unidad

Diagramas de clase

Autores: Ana Olsson Rodríguez, Sofía Calvo López y Duna Puente Romera

Grupo de laboratorio: 6

Fecha: 8 de abril de 2026

Tabla de contenidos

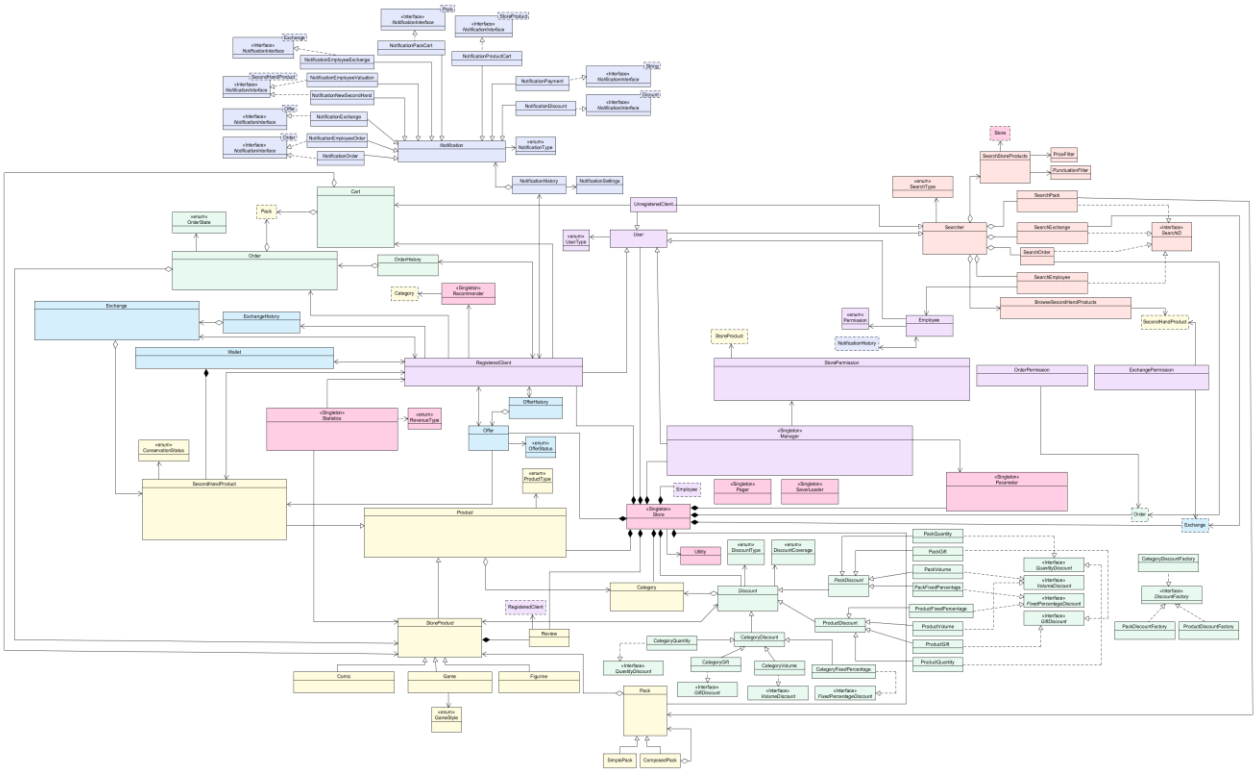
Indicación inicial....1
Diagrama de clases general....3
Diagrama: Paquete Store....4
Diagrama: Paquete Utility....5
Diagrama: Paquete User....6
Diagrama: Paquete Search....6
Diagrama: Paquete Product....7
Diagrama: Paquete Order....8
Diagrama: Paquete Notification....9
Diagrama: Paquete Exchange....10
Diagrama: Paquete Discount....11

Indicación inicial

A continuación, se incluyen las imágenes de los diagramas de clases en líneas generales y divididos por paquetes. Dado el tamaño de estos que impide poder ver de un vistazo las clases de un paquete, se ha creado una carpeta donde poder visualizar con mejor resolución, pudiendo hacer zoom y ver al detalle cada uno de los diagramas. Se recomienda visualizar el archivo desde Microsoft Edge.

Enlace: Diagramas PADSOF 3.1.

Diagrama de clases general



Leyenda

Diagrama: Paquete Store

Diagrama: Paquete Utility

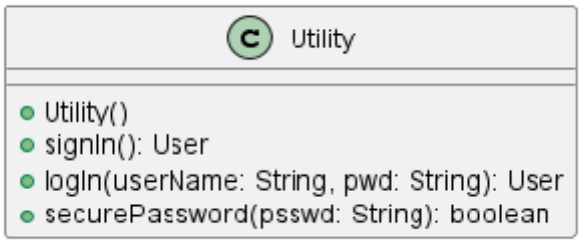


Diagrama: Paquete User

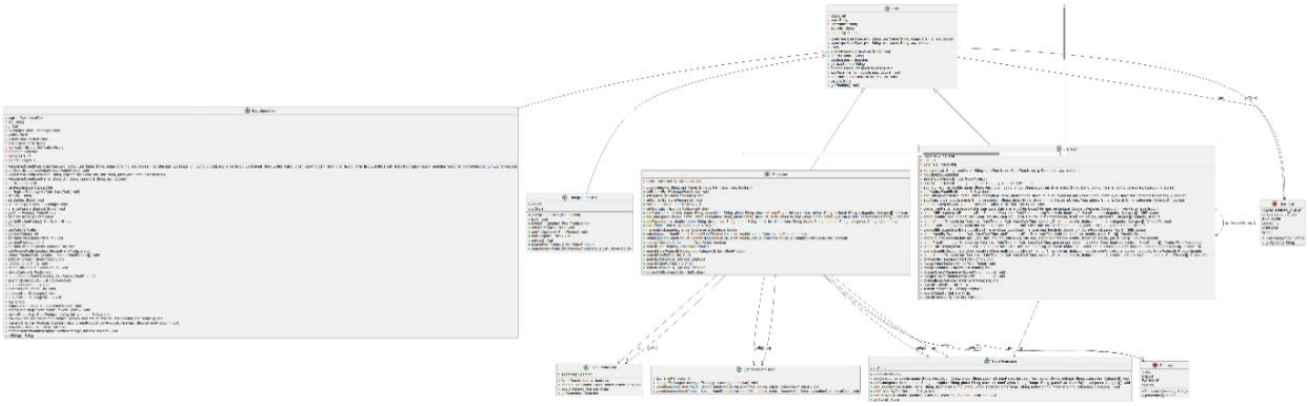


Diagrama: Paquete Search

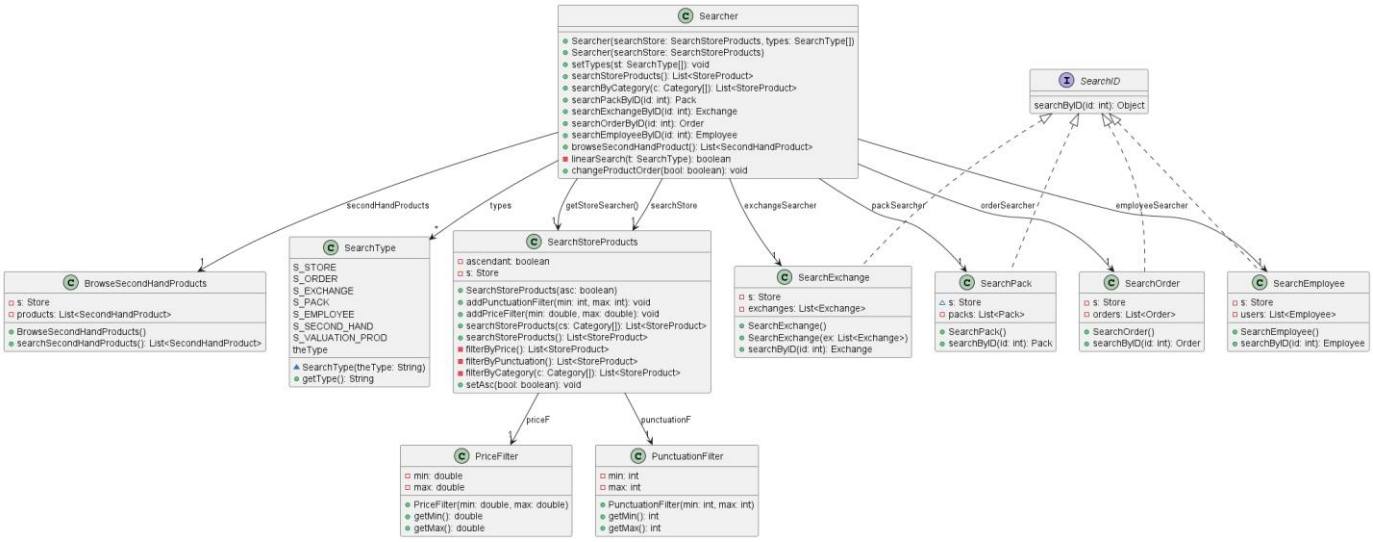


Diagrama: Paquete Product

Diagrama: Paquete Order

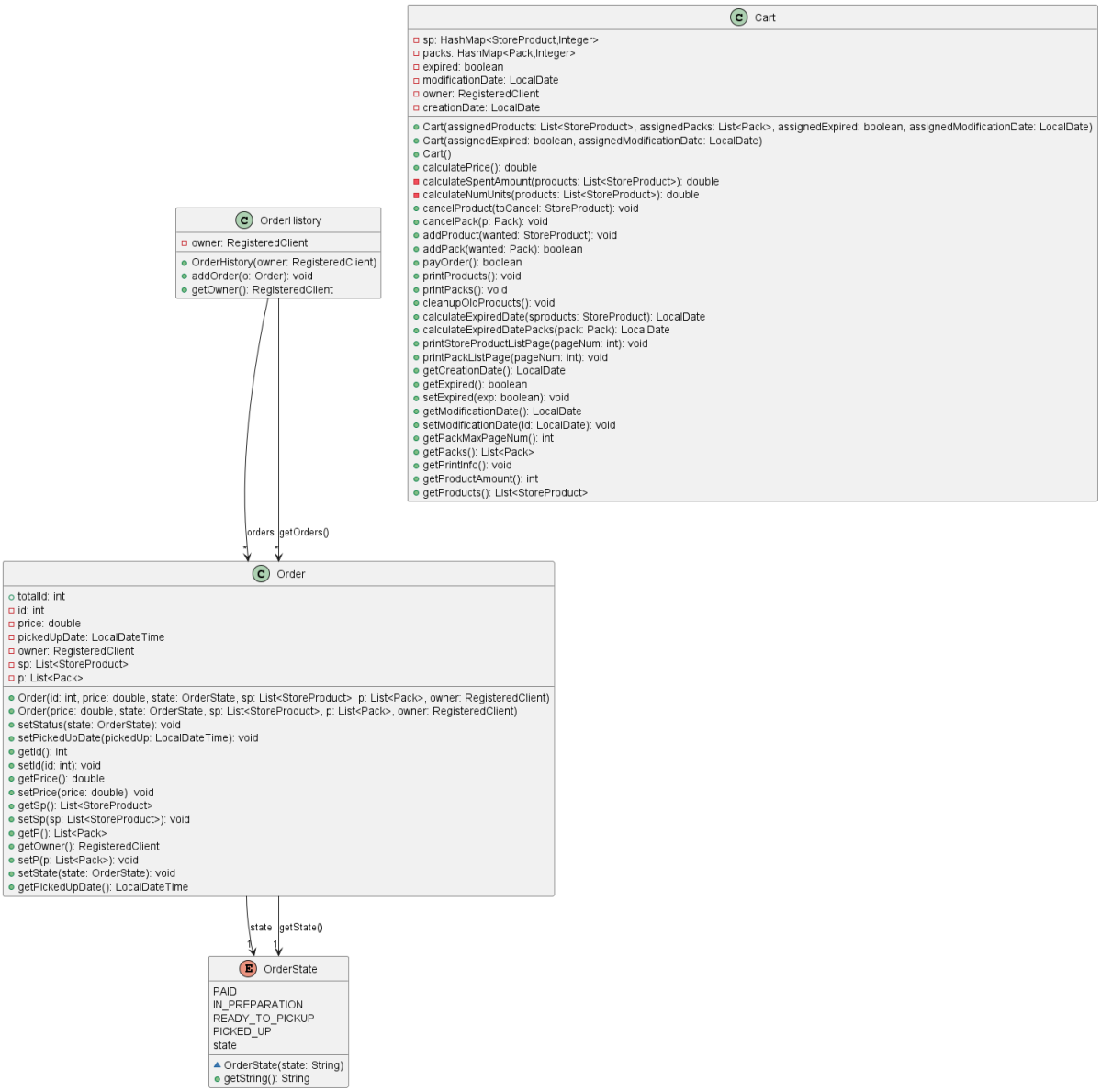

Cart
sp: HashMap<StoreProduct,Integer>
packs: HashMap<Pack,Integer>
expired: boolean
modificationDate: LocalDate
owner: RegisteredClient
creationDate: LocalDate
Cart(assignedProducts: List<StoreProduct>, assignedPacks: List<Pack>, assignedExpired: boolean, assignedModificationDate: LocalDate)
Cart(assignedExpired: boolean, assignedModificationDate: LocalDate)
Cart()
calculatePrice(): double
calculateSpentAmount(products: List<StoreProduct>): double
calculateNumUnits(products: List<StoreProduct>): double
cancelProduct(toCancel: StoreProduct): void
cancelPack(p: Pack): void
addProduct(wanted: StoreProduct): void
addPack(wanted: Pack): boolean
payOrder(): boolean
printProducts(): void
printPacks(): void
cleanupOldProducts(): void
calculateExpiredDate(sproducts: StoreProduct): LocalDate
calculateExpiredDatePacks(pack: Pack): LocalDate
printStoreProductListPage(pageNum: int): void
printPackListPage(pageNum: int): void
getCreationDate(): LocalDate
getExpired(): boolean
setExpired(exp: boolean): void
getModificationDate(): LocalDate
setModificationDate(ld: LocalDate): void
getPackMaxPageNum(): int
getPacks(): List<Pack>
getPrintInfo(): void
getProductAmount(): int
getProducts(): List<StoreProduct>
OrderHistory
owner: RegisteredClient
OrderHistory(owner: RegisteredClient)
addOrder(o: Order): void
getOwner(): RegisteredClient
orders
getOrders()
Order
totalId: int
id: int
price: double
pickedUpDate: LocalDateTime
owner: RegisteredClient
sp: List<StoreProduct>
p: List<Pack>
Order(id: int, price: double, state: OrderState, sp: List<StoreProduct>, p: List<Pack>, owner: RegisteredClient)
Order(price: double, state: OrderState, sp: List<StoreProduct>, p: List<Pack>, owner: RegisteredClient)
setStatus(state: OrderState): void
setPickedUpDate(pickedUp: LocalDateTime): void
getId(): int
setId(id: int): void
getPrice(): double
setPrice(price: double): void
getSp(): List<StoreProduct>
setSp(sp: List<StoreProduct>): void
getP(): List<Pack>
getOwner(): RegisteredClient
setP(p: List<Pack>): void
setState(state: OrderState): void
getPickedUpDate(): LocalDateTime
state
getState()
OrderState
PAID
IN_PREPARATION
READY_TO_PICKUP
PICKED_UP
state
OrderState(state: String)
getString(): String

Diagrama: Paquete Notification

Diagrama: Paquete Exchange

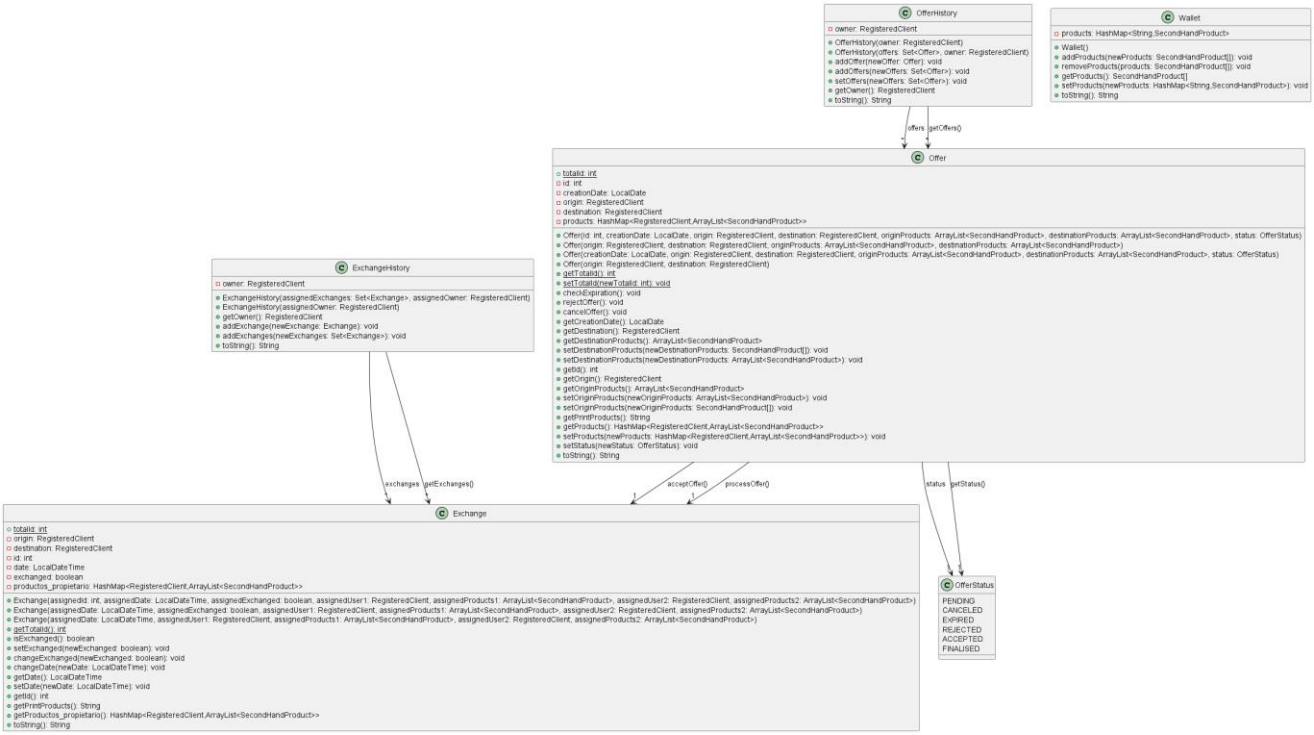


Diagrama: Paquete Discount